



SAKARYA
ÜNİVERSİTESİ

BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ

YAPAY ZEKAYA GİRİŞ DERSİ

Otonom Araç Simülasyonu için Basit Yapay Zeka Sistemi Geliştirme

Grup Üyeleri ve Numara:

Muhammet Emir AŞILI-B221200062

Yakup YOLCU- B211200011

Defne ÇİFÇİ- B231200032

Nimet Asude YALÇIN-B221200041

Raziyenur YANIK- B231200027

Sude BAŞALAN- B231200052

Umut Oktay TOPRAK-B221200044

Ayşe Nihal KAYMAK-B221200019

Zeynep Nur KAHYA-B231200381

Sena Zeynep GÖRGÜN- B231200054

Öğretim Üyesi

Dr. Esin Ayşe Zaimoğlu

2025-2026 Güz Proje/Tasarım Ödevi

1. GİRİŞ

Bu projede, bir otonom aracın simülasyon ortamında hedef konuma güvenli ve verimli bir şekilde ulaşmasını sağlamak amacıyla yapay zeka tabanlı bir yol bulma sistemi geliştirilmiştir. Simülasyon ortamı Python programlama dili kullanılarak oluşturulmuş; araç, engeller ve çevresel koşullar dikkate alınarak hareket edecek şekilde modellenmiştir.

Proje kapsamında üç farklı arama algoritması kullanılmıştır: Derinlik Öncelikli Arama, Genişlik Öncelikli Arama ve A* algoritması. DFS algoritması sezgisel olmayan ve kesin çözüm yaklaşımı olarak uygulanmış; BFS algoritması basit ve hızlı bir çözüm yöntemi olarak değerlendirilmiş; A* algoritması ise hesaplama maliyeti ve yol uzunluğu açısından optimal çözüm sunan bir yöntem olarak tercih edilmiştir.

Geliştirilen sistemin temel amacı, aracın önceden tanımlanmış bir harita üzerinde:

- Çevresindeki engelleri algılayarak çarpışmalardan kaçınmasını,
- Trafik kurallarına ve çevresel kısıtlamalara uygun şekilde hareket etmesini,
- Birden fazla rota arasından en uygun olanı seçerek hedef konuma ulaşmasını

sağlamaktır.

2. PROJE AMACI VE KAPSAMI

2.1 Proje Amacı

Bu projenin temel amacı, arama algoritmaları ve kural tabanlı karar verme sistemleri kullanılarak bir otonom araç simülasyonu geliştirmektir.

2.2 Proje Kapsamı

Proje kapsamında:

- Harita tabanlı bir simülasyon ortamı oluşturulmuştur.
- Araç hareketleri ve çevresel etkileşimler modellenmiştir.
- Üç farklı yapay zeka algoritması uygulanmıştır.
- Algoritmaların performansı test edilmiştir.

3. SİSTEM MİMARİSİ VE DOSYA YAPISI

Proje, modüler bir mimari ile geliştirilmiştir. Her bileşen farklı bir klasör altında toplanmıştır.

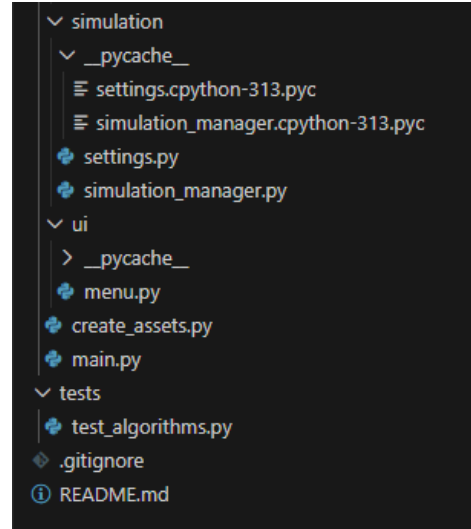
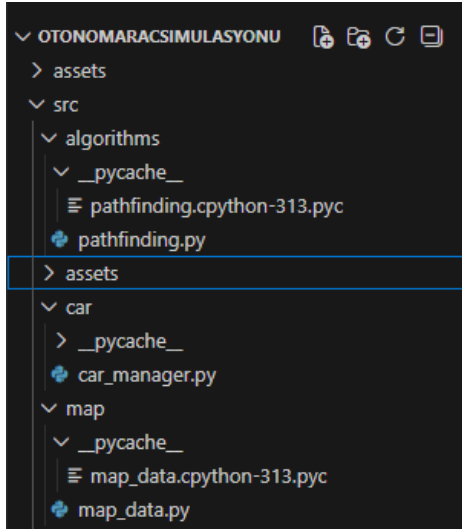
3.1 Genel Mimari

Sistem aşağıdaki ana bileşenlerden oluşmaktadır:

- Harita Yönetimi
- Araç Kontrolü
- Yapay Zeka Algoritmaları

- Simülasyon Yönetimi
- Kullanıcı Arayüzü

3.2 Dosya ve Klasör Yapısı



- **algorithms:** otonom aracın yol bulma ve karar verme süreçlerinde kullanılan yapay zekâ algoritmaları
- **car:** aracın hareketleri ve davranışları ile ilgili kodlar
- **map:** simülasyon ortamında kullanılan harita yapısını ve çevresel öğeler
- **simulation:** simülasyonun genel akışını ve zaman yönetimini kontrol eden kodlar
- **ui:** kullanıcı ile etkileşimi sağlayan arayüz bileşenleri
- **main.py:** Projenin çalıştırıldığı ana dosyadır. Gerekli modüller bu dosyada çağrılmakta ve simülasyon süreci başlatılmaktadır.
- **create_assets.py:** Simülasyonda kullanılan görsel veya yardımcı varlıkların oluşturulması ve düzenlenmesi amacıyla kullanılan yardımcı bir dosyadır.
- **test_algorithms.py:** DFS, BFS ve A* algoritmalarının doğruluğunu ve beklenen çıktıları üretip üretmediğini test etmek amacıyla oluşturulmuştur.

4. SİMÜLASYON ORTAMI

4.1 Giriş Ekranı



4.2 Harita Yapısı

Harita, hücre tabanlı bir yapı şeklinde modellenmiştir. Harita üzerinde:

- Başlangıç noktası
- İstanbul Sokakları ve Marmara Denizi
- Engeller
 - Çukur
 - Trafik Lambası
 - Yaya Geçidi
- Hedef noktası bulunmaktadır.

map_data.py

- 0 – Yol: Aracın serbestçe hareket edebildiği alanları temsil eder.
- 1 – Bina / Çim: Araç tarafından geçilemeyen engel alanlarıdır.
- 2 – Su (Kanal): Geçişe kapalı bölgeleri ifade eder ve doğal engel olarak değerlendirilir.
- 3 – Başlangıç Noktası: Otonom aracın simülasyona başladığı konumu temsil eder.
- 4 – Hedef Noktası: Aracın ulaşması gereken nihai konumu ifade eder.
- 5 – Kavşak: Birden fazla yol seçeneğinin bulunduğu yol ayrım noktalarını temsil eder.
- 6 – Trafik Işığı: Trafik kurallarını simüle etmek amacıyla kullanılan kontrol noktalarıdır.

Simülasyon, Pygame kütüphanesi kullanılarak gerçek zamanlı bir döngü mantığıyla çalıştırılmaktadır. Bu yapı sayesinde sistem, her karede güncellenerek dinamik bir simülasyon ortamı sunmaktadır.

4.3.1 Algoritma Seçimi ve Başlangıç Rotası

Kullanıcı, menü ekranından DFS, BFS veya A* algoritmalarından birini seçmektedir. Seçilen algoritmaya göre, başlangıç ve hedef noktaları arasında ilk rota hesaplanmakta ve araç bu rotayı izlemeye başlamaktadır.

4.3.2 Gerçek Zamanlı Simülasyon Döngüsü

Simülasyon, gerçek zamanlı bir döngü içerisinde çalışmaktadır. Her karede araç durumu, zaman bilgisi, çevresel etkenler ve ekran çizimleri güncellenmektedir. Bu yapı, simülasyonun akıcı ve dinamik olmasını sağlamaktadır.

4.3.3 Trafik Işıkları ve Çevresel Kurallar

Simülasyonda trafik ışıkları zaman tabanlı olarak kontrol edilmektedir. Araç, kırmızı ışıktaki durmakta ve yeşil ışıktaki hareketine devam etmektedir. Bu mekanizma, aracın trafik kurallarına uygun davranmasını sağlamaktadır.

4.3.4 Araç Hareketi ve Hız Kontrolü

Araç hareketi, yalnızca rota bilgisine değil çevresel koşullara da bağlıdır. Araç; normal yollarda sabit hızla ilerlemekte, yaya geçitlerinde hızını düşürmekte ve durması gereken durumlarda tamamen durmaktadır.

4.3.5 Dinamik Engel Oluşturulması

Simülasyon ilerledikçe aracın rotası üzerine dinamik bir engel yerleştirilmektedir. Bu engel başlangıçta haritada bulunmamakta ve simülasyon sırasında ortaya çıkmaktadır.

4.3.6 Yeniden Yol Planlama

Dinamik engel oluştuğunda araç, mevcut rotasını terk ederek bulunduğu konumdan hedefe yeni bir rota hesaplamaktadır. Bu işlem, seçili olan DFS, BFS veya A* algoritması kullanılarak gerçekleştirilmektedir.

4.3.7 Performans Ölçümü

Simülasyon boyunca algoritmaların performansını değerlendirmek amacıyla toplam süre, kare sayısı ve yol uzunluğu gibi metrikler ölçülmektedir. Bu veriler, algoritmaların karşılaştırılmasında kullanılmaktadır.

4.3.8 Görselleştirme ve Kullanıcı Bilgilendirme

Simülasyon ekranında aktif algoritma bilgisi, araç hızı ve izlenen rota görsel olarak gösterilmektedir. Bu sayede kullanıcı, simülasyon sürecini anlık olarak takip edebilmektedir.

4.4 Araç Hareketi ve Davranış Modeli

Otonom aracın simülasyon ortamındaki tüm hareketleri, hız kontrolü ve çevresel tepkileri car_manager.py dosyası içerisinde tanımlanmıştır. Bu dosya, aracın yol bulma algoritmalarından elde

ettiği rotayı nasıl takip ettiğini ve çevresel koşullara göre nasıl davranış sergilediğini kontrol eden temel bileşendir.

Araç, hücre tabanlı bir harita üzerinde konumlandırılmakta ve hem grid koordinatları hem de piksel koordinatları kullanılarak akıcı bir şekilde hareket ettirilmektedir.

Araç;

- Karşısına çıkan çukur engellerini algılar ve rota değişikliği yapar.
- Trafik ışığıyla karşılaştığında; kırmızıda durur, yeşilde geçer.
- Yaya geçidine geldiğinde hızını düşürür.
-

Araç Hareketinin Genel Yönetimi

Araç; konum bilgisi, hız kontrolü ve çevresel tepkilerinin tamamı `car_manager.py` dosyası üzerinden yönetilmektedir. Bu dosya, aracın simülasyon ortamında nasıl hareket edeceğini belirleyen ana modüldür.

Konulandırma ve Görsel Temsil

Araç, hücre tabanlı harita üzerinde satır-sütun bilgileri ile takip edilmekte ve piksel koordinatları kullanılarak akıcı bir şekilde hareket ettirilmektedir. Hareket yönüne bağlı olarak araç görseli döndürülmektedir.

Rota Takibi

Araç, DFS, BFS veya A* algoritmaları tarafından hesaplanan rotayı adım adım takip etmektedir. Her hedef hücreye ulaşıldığında bir sonraki hücreye geçilmektedir.

Trafik Işıkları ile Etkileşim

Araç, bulunduğu hücrede kırmızı trafik ışığı tespit ettiğinde durmakta, yeşil ışık durumunda hareketine devam etmektedir. Bu yapı, trafik kurallarına uygun sürüşü simüle etmektedir.

Dinamik Engel Algılama

Araç, rotası üzerinde bir sonraki adımda dinamik engel olup olmadığını kontrol etmektedir. Engel tespit edildiğinde hareket durdurulmakta ve sistem yeniden rota planlamaya yönlendirilmektedir.

Hız Kontrolü ve Çevresel Etkenler

Araç hızı; yol türüne, kavşaklara, trafik ışıklarına ve yaya geçitlerine bağlı olarak dinamik biçimde ayarlanmaktadır. Yaya geçitlerinde hız düşürülmekte, gerekli durumlarda araç tamamen durmaktadır.

Akıcı Hareket ve Yön Değişimi

Araç, hedef hücreye doğru yön ve mesafe hesaplamaları yaparak ilerlemekte ve hareket yönüne uygun şekilde döndürülmektedir. Bu yaklaşım, gerçekçi bir sürüş deneyimi sunmaktadır.

Ekrana Çizim

Araç, her karede güncellenen konumuna göre ekran üzerine çizdirilmektedir. Bu sayede araç hareketleri kesintisiz ve net bir şekilde gözlemlenebilmektedir.

5. YOL BULMA ALGORİTMALARI VE UYGULAMASI

Bu projede, otonom aracın başlangıç noktasından hedefe ulaşabilmesi için üç farklı yol bulma algoritması kullanılmıştır: Genişlik Öncelikli Arama (BFS), Derinlik Öncelikli Arama (DFS) ve A* algoritması. Tüm algoritmalar, hücre tabanlı harita yapısı üzerinde çalışacak şekilde tasarlanmış ve dinamik engelleri dikkate alacak biçimde uygulanmıştır.

Yürünebilir Alan ve Komşuluk Yapısı

Algoritmaların çalışabilmesi için harita üzerinde aracın hareket edebileceği hücreler önceden tanımlanmıştır. Yol, kavşak, trafik ışığı ve yaya geçidi hücreleri yürünebilir olarak kabul edilmiştir. Her hücre için yukarı, aşağı, sağ ve sol yönlerde komşular kontrol edilerek geçerli komşu hücreler belirlenmiştir. Ayrıca dinamik engeller, komşuluk kontrolü sırasında dikkate alınarak algoritmaların bu hücrelerden kaçınması sağlanmıştır.

5.1 Genişlik Öncelikli Arama (BFS)

BFS algoritması, başlangıç noktasından itibaren tüm komşu hücreleri katman katman inceleyerek çalışmaktadır. Bu yaklaşım sayesinde BFS, hedefe ulaşan en kısa adım sayısına sahip yolu garanti eder. Ancak geniş alanlarda daha fazla düğüm ziyaret ettiği için hesaplama maliyeti yüksektir.

Bu projede BFS, basit ve hızlı çözüm yaklaşımı olarak kullanılmıştır.



Şekil 5.1: BFS algoritması ile elde edilen rota

5.2 Derinlik Öncelikli Arama (DFS)

DFS algoritması, başlangıç noktasından itibaren bir yolu mümkün olduğunca derinlemesine takip eder. Hedef bulunamazsa geri dönerek alternatif yolları dener. DFS, her zaman en kısa yolu bulmayı garanti etmez ancak **kesin bir çözüm üretir**.

Bu projede DFS, **sezgisel olmayan ve kesin çözüm yaklaşımı** olarak uygulanmıştır.



Şekil 5.2: DFS algoritması ile elde edilen rota

5.3 A* Algoritması

A* algoritması, hem mevcut yol maliyetini hem de hedefe olan tahmini uzaklığı (Manhattan sezgisel fonksiyonu) dikkate alarak çalışmaktadır. Bu sayede gereksiz düğümleri ziyaret etmeden, en kısa ve en verimli rotayı bulmayı amaçlamaktadır.

Bu projede A*, optimal çözüm yaklaşımı olarak kullanılmıştır ve dinamik engeller sonrası yeniden rota hesaplamada da etkin şekilde uygulanmıştır.



Şekil 5.3: A* algoritması ile elde edilen optimal rota

Dinamik Engellerin Algoritmalar Etkisi

Tüm algoritmalar, yol hesaplama sırasında dinamik engelleri dikkate alacak şekilde geliştirilmiştir. Simülasyon sırasında yeni bir engel ortaya çıktığında, algoritmalar mevcut konumdan hedefe yeniden bir rota hesaplamaktadır. Bu sayede algoritmaların dinamik ortamlara adaptasyon yetenekleri gözlemlenebilmektedir.

Algoritmaların Karşılaştırılması

- **BFS:** En kısa yolu bulur, ancak daha fazla düğüm ziyaret eder.
- **DFS:** Kesin çözüm üretir, ancak yol optimal olmayabilir.
- **A*:** En kısa ve en verimli yolu genellikle en hızlı şekilde bulur.

Bu karşılaştırma, algoritmaların performans ve çözüm kalitesi açısından değerlendirilmesine olanak sağlamaktadır.

6. GÖRSEL ASSETS VE GÖRSELLEŞTİRME YAPISI

Tüm görsel dosyalar, proje dizini altında yer alan assets klasöründe toplanmıştır. Bu yapı sayesinde simülasyonda kullanılan görseller merkezi bir şekilde yönetilmiştir.

Harita ve Zemin Görselleri

Simülasyon ortamındaki harita; yol, kavşak, bina, su kanalı, yaya geçidi ve trafik ışıkları gibi farklı hücre türlerini görsel olarak ayırt edebilecek şekilde tasarlanmıştır. Bu amaçla:

- Yol ve kavşak görselleri, farklı yön ve bağlantı tiplerine göre ayrı ayrı tanımlanmıştır.
- Bina ve su alanları geçilemez bölgeleri temsil edecek şekilde görselleştirilmiştir.
- Yaya geçitleri özel zemin görselleri ve çizgilerle belirgin hale getirilmiştir.

Bu görseller, harita yapısındaki hücre türlerine göre dinamik olarak ekrana çizdirilmektedir.

Araç Görseli



Otonom araç, simülasyon içerisinde tek bir araç görseli kullanılarak temsil edilmiştir. Araç görseli:

- Harita hücre boyutlarına uygun şekilde ölçeklendirilmiştir.
- Araç hareket yönüne bağlı olarak döndürülerek gerçekçi bir sürüş hissi oluşturulmuştur.

Bu sayede aracın hangi yöne doğru ilerlediği kullanıcı tarafından kolaylıkla takip edilebilmektedir.

Trafik Işıkları ve Yaya Geçidi Görselleri



Trafik ışıkları, aktif ışık durumunu gösteren dinamik çizimler ile modellenmiştir. Kırmızı ve yeşil ışıklar, simülasyon sırasında zaman tabanlı olarak güncellenmektedir.

Yaya geçitleri ise özel zemin görselleri ve çizgi desenleri kullanılarak yol üzerinde açık bir şekilde vurgulanmıştır. Bu alanlarda araç hızının düşürülmesi, görsel olarak da desteklenmiştir.

Dinamik Engel Görselleri



Simülasyon sırasında ortaya çıkan dinamik engeller, özel bir engel görseli ile temsil edilmiştir. Bu engeller:

- Başlangıçta haritada bulunmamakta,
- Simülasyon ilerledikçe araç rotası üzerine eklenmektedir.

Bu görselleştirme, yeniden yol planlama mekanizmasının kullanıcı tarafından kolayca fark edilmesini sağlamaktadır.

7. DENEYSEL ÇALIŞMALAR VE PERFORMANS ÖLÇÜMÜ

Bu çalışmada, geliştirilen otonom araç simülasyonunda kullanılan BFS, DFS ve A* algoritmalarının performansları ölçülmüş ve karşılaştırılmıştır. Performans değerlendirmesi, simülasyon sırasında toplanan nicel veriler üzerinden gerçekleştirilmiştir.

Simülasyon her algoritma için **birden fazla kez çalıştırılmış** ve aşağıdaki metrikler ölçülmüştür:

- Toplam simülasyon süresi (saniye)
- İşlenen toplam kare (frame) sayısı
- Aracın kat ettiği toplam yol uzunluğu (birim)

Bu ölçümler, SimulationMetrics sınıfı kullanılarak otomatik olarak elde edilmiştir.

8. PERFORMANS ÖLÇÜM METODOLOJİSİ

Simülasyon başladığında zaman ölçümü başlatılmış, her karede araç konumu izlenmiş ve aracın kat ettiği mesafe hücre bazlı olarak hesaplanmıştır. Araç hedef noktaya ulaştığında simülasyon sonlandırılmış ve performans değerleri konsol çıktısı olarak raporlanmıştır.

Bu yaklaşım sayesinde algoritmalar:

- Hız (çözüm süresi),
- İşlem yoğunluğu (kare sayısı),
- Yol verimliliği (yol uzunluğu)

açısından objektif olarak karşılaştırılabilmektedir.

9. ALGORİTMA PERFORMANS KARŞILAŞTIRMASI VE SONUÇLAR

Aşağıdaki tabloda, her algoritmanın birden fazla çalıştırma sonucunda elde edilen tipik değerleri gösterilmektedir.

9.1 Performans Karşılaştırma Tabloları

Algoritma	BFS	DFS	A*
Toplam süre	19,48	35,23	19,02
Toplam Kare	1182	2131	1153
Yol Uzunluğu	85 birim	167 birim	71 birim

Algoritma	BFS	DFS	A*
Toplam süre	20,22	35,87	21,12
Toplam Kare	1221	2165	1275
Yol Uzunluğu	85 birim	159 birim	85 birim

Algoritma	BFS	DFS	A*
Toplam süre	20,51	36,03	18,09
Toplam Kare	1245	2188	1098
Yol Uzunluğu	85 birim	159 birim	75 birim

Tablo değerleri, simülasyonun farklı çalıştırmalarında elde edilen sonuç aralıklarını göstermektedir.

9.2 Sonuçların Değerlendirilmesi

BFS Algoritmasının Değerlendirilmesi

BFS algoritması, tüm çalıştırmalarda 85 birimlik sabit yol uzunluğu üretmiştir. Bu durum, BFS'in en kısa yolu garanti eden yapısıyla uyumludur. Ancak BFS, her ne kadar yol uzunluğu açısından tutarlı sonuçlar üretse de, toplam kare sayısının 1182–1245 aralığında değişmesi, algoritmanın geniş alanları taramasından kaynaklanan işlem maliyetini göstermektedir. Toplam süre değerlerinin 19–21 saniye aralığında olması, BFS'in makul bir çözüm süresi sunduğunu ortaya koymaktadır.

DFS Algoritmasının Değerlendirilmesi

DFS algoritması, tüm çalıştırmalarda en uzun yol uzunluklarını üretmiştir. Yol uzunluğu değerlerinin 159–167 birim aralığında değişmesi, DFS'in optimal yol garantisi sunmayan yapısını açıkça ortaya koymaktadır. Ayrıca toplam kare sayısının 2131–2188 gibi yüksek değerlerde olması, DFS'in daha fazla düğüm ziyaret ettiğini ve dolayısıyla daha yüksek işlem maliyetine sahip olduğunu göstermektedir. Toplam süre değerlerinin yaklaşık 35–36 saniye olması, DFS'in bu simülasyon için en verimsiz algoritma olduğunu ortaya koymaktadır.

A* Algoritmasının Değerlendirilmesi

A* algoritması, genel olarak en kısa ve en verimli rotaları üretmiştir. Yol uzunluğu bazı çalıştırmalarda BFS ile aynı değeri alsa da, diğer çalıştırmalarda 71 ve 75 birim gibi daha kısa yollar üretmiştir. Toplam kare sayısının 1098–1275 aralığında olması, A* algoritmasının BFS ve DFS'e kıyasla daha az düğüm işlediğini göstermektedir. Toplam süre değerlerinin 18–21 saniye aralığında olması, A* algoritmasının hem hızlı hem de optimal bir çözüm sunduğunu doğrulamaktadır.

Algoritmaların Genel Karşılaştırması

Elde edilen veriler ışığında:

- BFS, kısa yol garantisi sunmasına rağmen işlem maliyeti açısından orta seviyede kalmıştır.
- DFS, hem yol uzunluğu hem de işlem süresi açısından en zayıf performansı göstermiştir.
- A* algoritması ise düşük işlem maliyeti, kısa yol uzunluğu ve hızlı çözüm süresi ile en dengeli ve verimli algoritma olarak öne çıkmıştır.

Bu sonuçlar, A* algoritmasının sezgisel yapısı sayesinde gereksiz aramaları azalttığını ve simülasyon ortamına daha uygun bir çözüm sunduğunu göstermektedir.

10. SONUÇ VE DEĞERLENDİRME

Bu projede, bir otonom araç için temel seviyede yapay zekâ tabanlı bir simülasyon sistemi başarıyla geliştirilmiştir. Araç, hücre tabanlı bir harita üzerinde çevresel etkenleri dikkate alarak başlangıç noktasından hedefe güvenli ve kurallara uygun şekilde ilerleyebilmektedir.

Proje kapsamında DFS, BFS ve A* yol bulma algoritmaları uygulanmış ve bu algoritmalar toplam süre, işlenen kare sayısı ve yol uzunluğu gibi metrikler üzerinden karşılaştırılmıştır. Elde edilen sonuçlar, DFS algoritmasının optimal olmayan ve yüksek maliyetli çözümler ürettiğini, BFS algoritmasının en kısa yolu garanti etmesine rağmen daha fazla düğüm ziyaret ettiğini göstermiştir. A* algoritması ise sezgisel yapısı sayesinde daha kısa ve verimli rotalar üreterek en dengeli performansı sergilemiştir.

Simölasyona trafik ışıkları, yaya geçitleri ve dinamik engeller entegre edilerek aracın çevresel koşullara uyum sağlayabilmesi hedeflenmiştir. Sonuç olarak geliştirilen sistem, otonom araçların temel yol planlama ve karar verme süreçlerini başarıyla modellemekte olup, ileride daha karmaşık haritalar ve gerçek zamanlı verilerle geliştirilmeye açıktır.